VIT-AP University, G-30, Inavolu, Beside AP Secretariat Amaravati, Andhra Pradesh 522237

Tiltle

Earthquake Detector

Submitted by :

1) Boddu SaiGanesh Reddy :22BCE8572
2) Aditya Shivnuri : 22BCE9354
3) Snehal Gullapudi : 22BCE7632
4) Karthikeya : 22BEC7387
5) Koojitha : 22BCE7087

Under the guidance of:

Prof.Sheela Jayachandran,

Department of Software and System Engineering,

VIT-AP University, Amaravati, Andhra Pradesh

Index:

Abstract	3
Introduction	4
Background	4
Problem Definition	5
Objectives	5
Methodology/Procedure	5
Results and Discussion	7
Conclusion and Future Scope	11
References	11
Appendix	12

Abstract:

Introducing an innovative Arduino Earthquake Detector Alarm with Seismic Graph project, designed for real-time earthquake detection and visualization. Utilizing an accelerometer, the Arduino board accurately measures ground acceleration, triggering alerts upon seismic activity surpassing preset thresholds. Employing advanced algorithms, non-earthquake vibrations are filtered out for precise detection. Additionally, a companion code in Processing IDE generates dynamic seismic graphs on a computer, providing visual insights into ground motion intensity over time. Users can experience the system's functionality by uploading the Arduino code, executing the Processing IDE code, and observing the graph's response to accelerometer-induced vibrations. This project amalgamates hardware and software for efficient earthquake monitoring and analysis.

Introduction

Earthquakes, natural phenomena resulting from the sudden release of energy in the Earth's crust, pose significant threats to lives and infrastructure worldwide. With their unpredictable nature and potentially devastating consequences, earthquakes can cause widespread destruction, including building collapses, infrastructure damage, and loss of life. Understanding and monitoring seismic activity is crucial for mitigating these risks and enhancing disaster preparedness.

The Arduino Earthquake Detector Alarm with Seismic Graph project presents a proactive solution to earthquake detection and analysis. By leveraging an accelerometer-based sensor system, the project enables real-time monitoring of ground acceleration, a key indicator of seismic activity. Upon detecting earthquake vibrations exceeding preset thresholds, the system triggers timely alerts, allowing for prompt response and evacuation measures.

Furthermore, the integration of sophisticated algorithms ensures accurate detection by filtering out non-earthquake vibrations, reducing false alarms and enhancing reliability. The companion code in Processing IDE complements this functionality by visualizing seismic data through dynamic graphs, facilitating in-depth analysis of ground motion intensity over time.

This comprehensive approach to earthquake monitoring and visualization empowers users with valuable insights into seismic events, enabling informed decision-making and efficient resource allocation during emergency situations. Ultimately, the project serves as a vital tool

for enhancing earthquake preparedness, minimizing damage, and safeguarding lives and infrastructure in earthquake-prone regions.

Background

1. Earthquake Detection Challenges: According to the United States Geological Survey (USGS), traditional earthquake detection methods face challenges such as limited coverage, delayed alerts, and false alarms. These limitations hinder timely response efforts and accurate assessment of seismic events.
2. Advancements in Sensor Technology: With advancements in sensor technology, particularly the development of low-cost accelerometers and microcontrollers like Arduino, there has been a growing interest in creating DIY earthquake detection systems. These systems leverage the affordability and accessibility of components to improve earthquake monitoring capabilities, enabling individuals and communities to better prepare for seismic events.

Problem Definition

Existing earthquake detection systems suffer from delayed alerts and inaccurate readings. This project addresses these issues by developing a real-time detection system using accelerometers and advanced algorithms. Additionally, it provides dynamic seismic graphs for intuitive visualization, aiming to enhance disaster preparedness in earthquake-prone areas.

Objectives

1. Develop a robust earthquake detection system utilizing accelerometer-based sensors and advanced algorithms to ensure real-time and accurate detection of seismic activity.
2. Implement threshold-based triggering mechanisms to promptly alert users when earthquake vibrations exceed preset thresholds, enabling timely response and evacuation procedures.
3. Integrate data visualization tools, such as dynamic seismic graphs generated using Processing IDE, to provide intuitive and insightful representations of ground motion intensity over time.
4. Enhance the system's reliability and effectiveness by implementing filters to differentiate between earthquake vibrations and non-earthquake disturbances, thereby reducing false alarms.
5. Conduct rigorous testing and validation procedures to evaluate the system's performance under various seismic conditions, ensuring its efficacy in enhancing disaster preparedness and response efforts in earthquake-prone regions.

Methodology/Procedure

Install the Arduino IDE

Download the Arduino IDE (Integrated Development Environment) from the official Arduino website https://www.arduino.cc/en/software

Select the appropriate version for your operating system (Windows, macOS, or Linux).

Follow the installation instructions for your Uno.

Connect Your Arduino Board:

Plug in your Arduino board to your computer using a USB cable.

Your computer should automatically recognize the Arduino board and install the necessary drivers. If not, you might need to manually install drivers from the Arduino website

Launch the Arduino IDE:

Open the Arduino IDE that you installed in step 1.

Configure the IDE:

Go to "Tools" and select your board model (e.g., Arduino Uno, Arduino Mega, etc.).

Choose the correct port in the "Port" menu. It should be the one where your Arduino is connected.

Test Your Setup:

Open a simple example sketch from "File" -> "Examples" -> "01.Basics" (e.g., Blink).

Click the "Upload" button (a right-arrow icon) to upload the sketch to your Arduino.

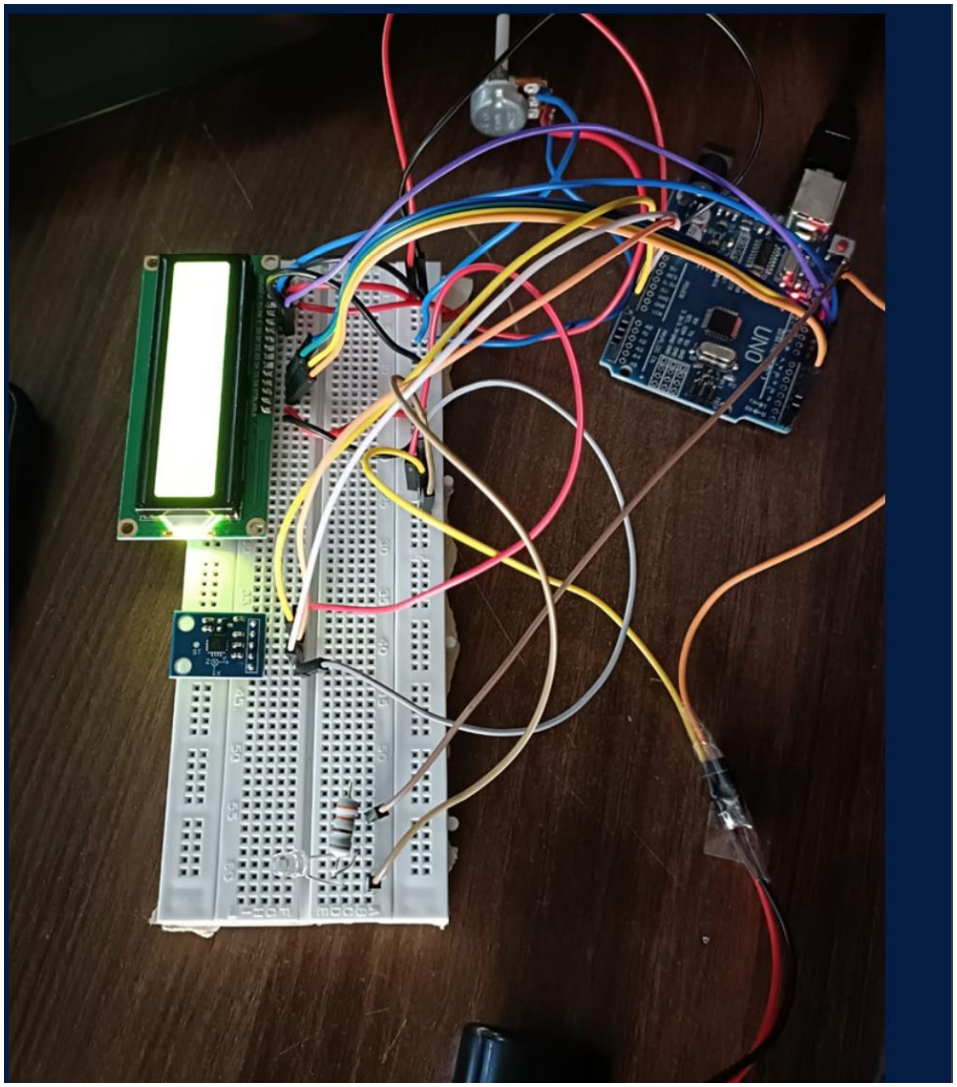
If everything is set up correctly, you should see the onboard LED blinking.

1. Arduino UNO is a microcontroller board based on ATmega328p. It has 20pins out of which 16 digital input and output pins and 6 analog input pins, 16MHZ Quartz crystal, power jack, ICSP header and reset button. It is very easy to perform with arduino since it is user friendly, The Operation Voltage is 5V, we can directly connect it to computer with USB cable, power it with AC-DC adapter or battery.

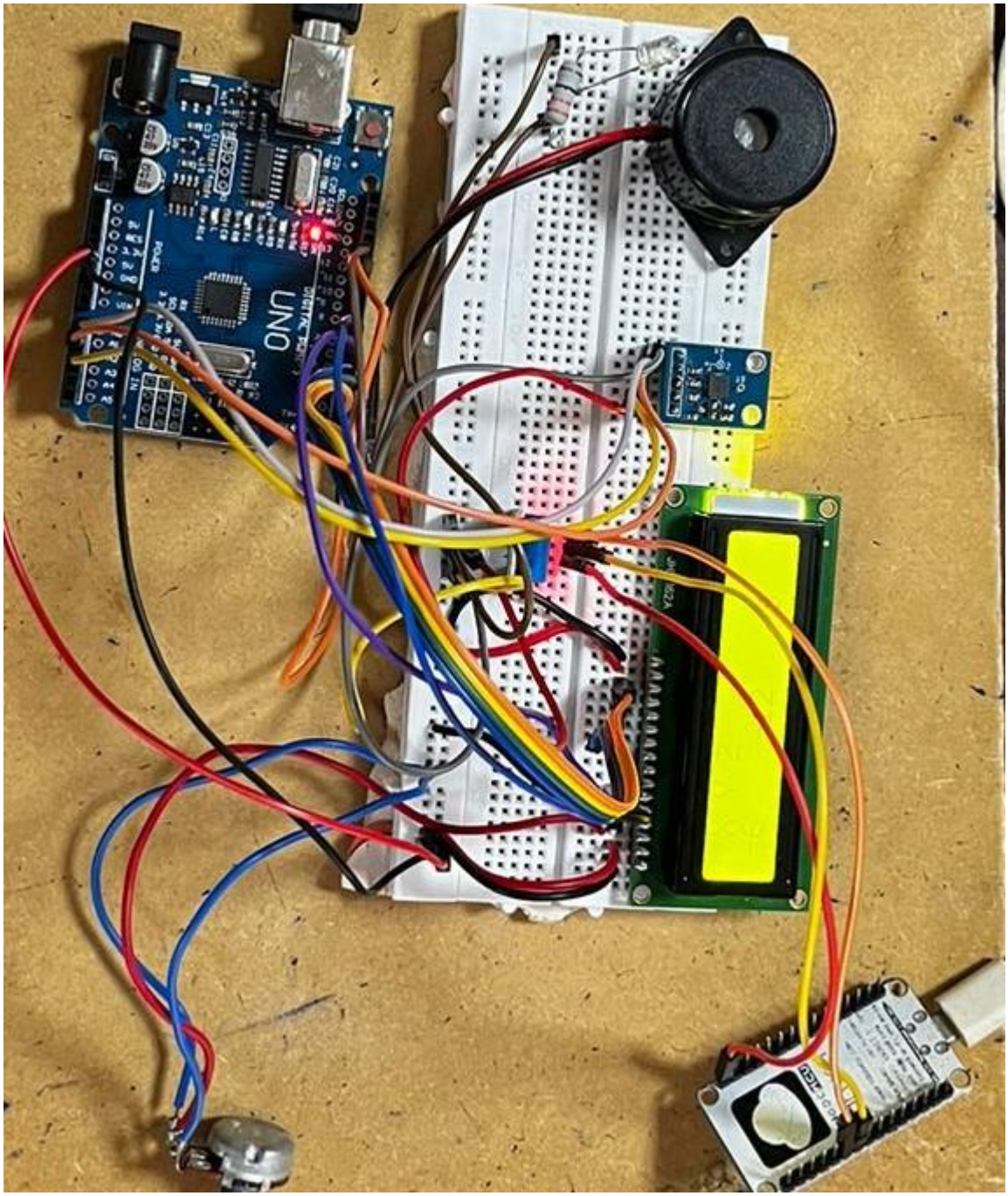
2. HC-SRC04 ultrasonic sensor has 4 pins-ground, Vcc, trigger and Echo. It ranging from 2cm to 500cm(5m). Mainly it has two opening –one is transmitter which is used to transmit the signal and another one is receiver which is used to receive the signal. It sends ultrasound waves at high frequency and receive back the signal.
3. Connect the ultrasonic sensors to the Arduino UNO. The input pins of trigger and echo of front ultrasonic sensor The buzzers are connected to pins 5 and 7. The buzzers are of different frequency and generate different sounds. First, we should calculate the distance of the obstacle with the help of ultrasonic sensor, which is Distance = (time taken * 0.034)/2
4. Whenever the sensor detects any object in its path the buzzer starts beeping and also at the same time the LED turns on.
5. The blind person can hear the beeping of the buzzer and manage to change the way. In this way, the person can easily find his way without getting injured.

Results and Discussion:

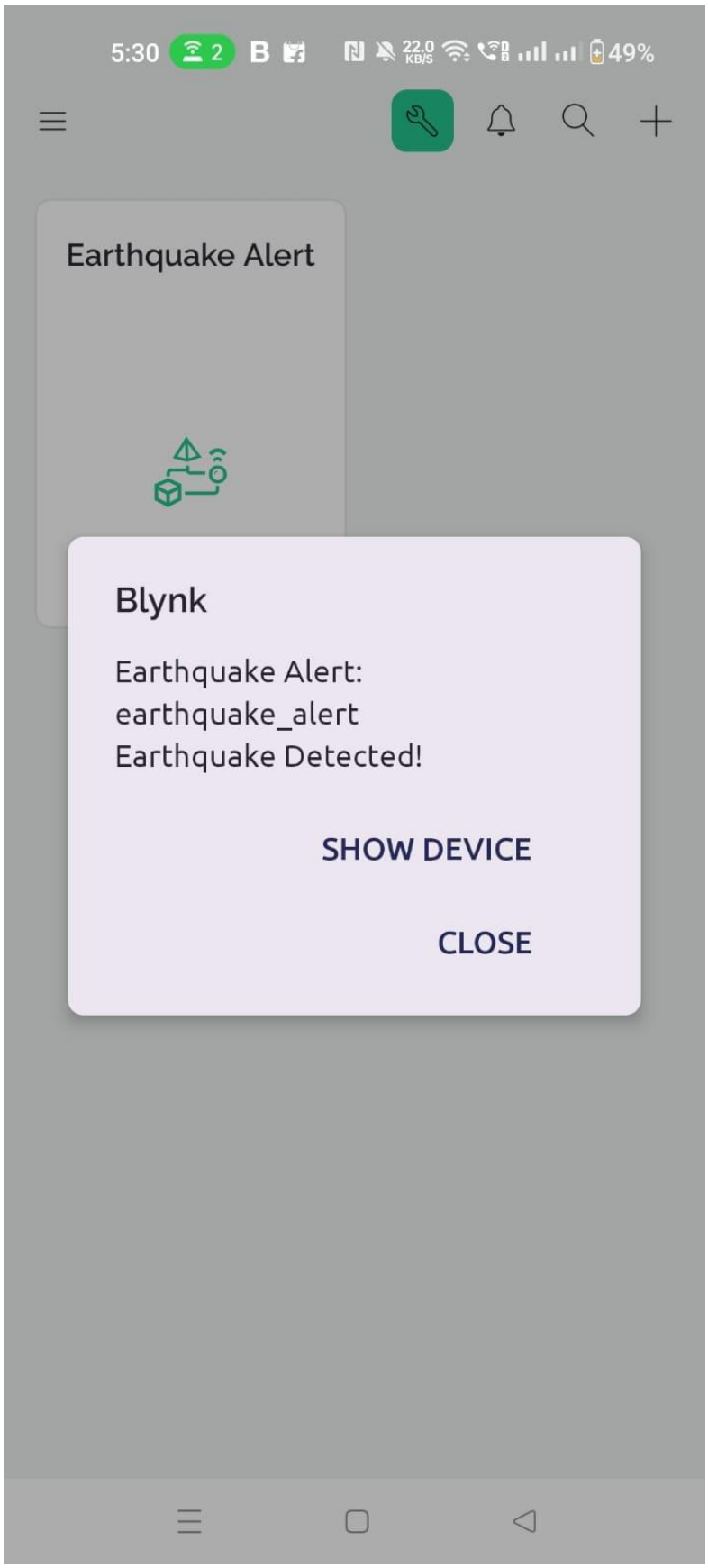
1)Arduino

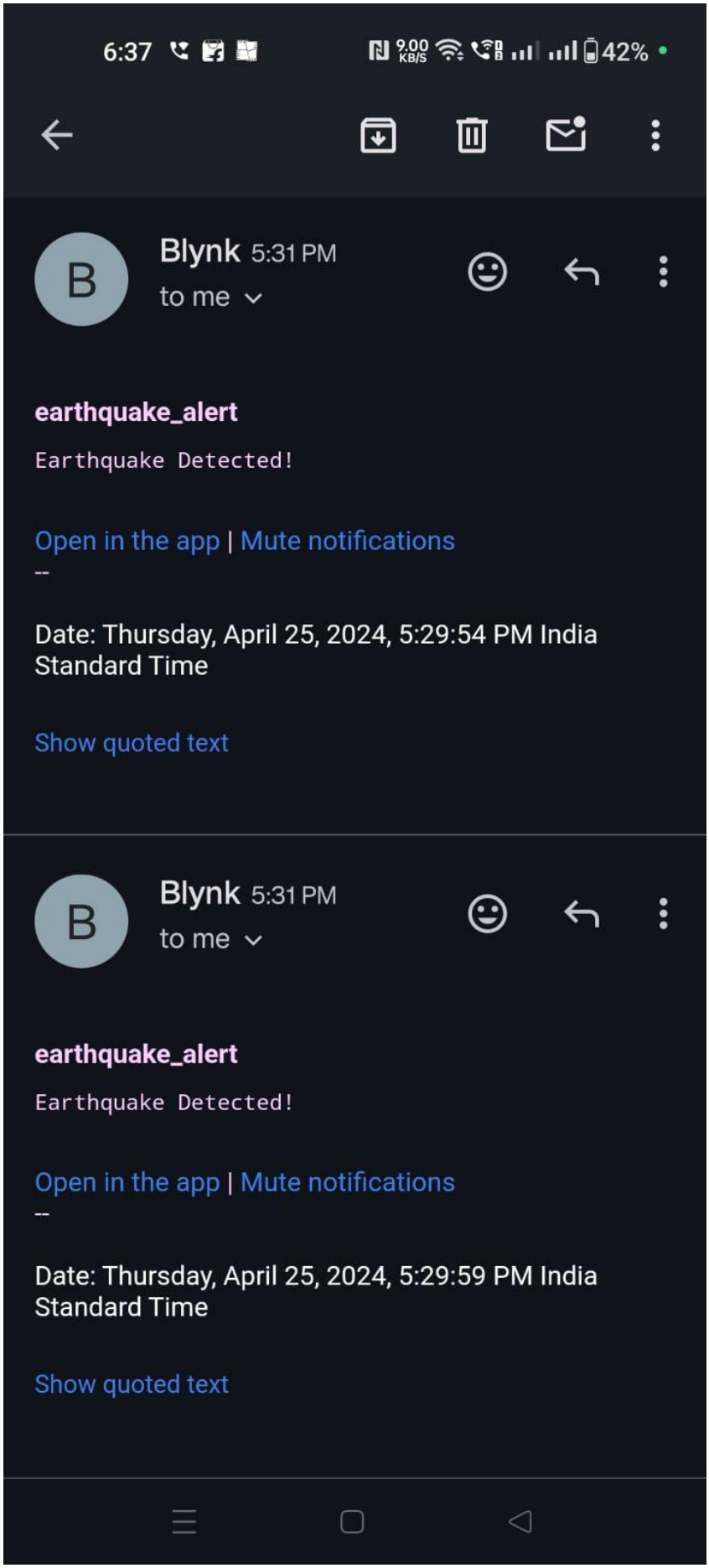


2)Proto Type:



3)Alert Message:




6:37
42%
Blynk 5:31 PM
to me
earthquake_alert
Earthquake Detected!
Open in the app | Mute notifications
--
Date: Thursday, April 25, 2024, 5:29:54 PM India Standard Time
Show quoted text
Blynk 5:31 PM
to me
earthquake_alert
Earthquake Detected!
Open in the app | Mute notifications
--
Date: Thursday, April 25, 2024, 5:29:59 PM India Standard Time
Show quoted text

Conclusion and Future Scope:

The Arduino Earthquake Detector Alarm with Seismic Graph project offers a holistic solution for real-time earthquake detection and visualization. Through accelerometer sensors, advanced algorithms, and dynamic seismic graphs, it enhances disaster preparedness with accurate alerts and intuitive data visualization. Future iterations could integrate machine learning for better disturbance distinction, wireless communication for remote monitoring, and collaboration with experts for predictive models. Continuous research and innovation promise to advance earthquake detection technology, bolstering resilience to seismic events and improving response efforts in vulnerable regions.

Appendix

Java code of Arduino

```
#include<LiquidCrystal.h> // lcd Header

LiquidCrystal lcd(7,6,5,4,3,2); // pins for LCD Connection


#define buzzer 12 // buzzer pin

#define led 13 //led pin


#define x A0 // x_out pin of Accelerometer

#define y A1 // y_out pin of Accelerometer

#define z A2 // z_out pin of Accelerometer


/*variables*/

int xsample=0;

int ysample=0;

int zsample=0;

long start;

int buz=0;
```

```
/*Macros*/
#define samples 50
#define maxVal 20 // max change limit
#define minVal -20 // min change limit
#define buzTime 5000 // buzzer on time
 void setup()
{
lcd.begin(16,2); //initializing lcd
Serial.begin(9600); // initializing serial
delay(1000);
lcd.print("EarthQuake ");
lcd.setCursor(0,1);
lcd.print("Detector ");
delay(2000);
lcd.clear();
lcd.print("Calibrating.....");
lcd.setCursor(0,1);
lcd.print("Please wait...");
pinMode(buzzer, OUTPUT);
pinMode(led, OUTPUT);
buz=0;
digitalWrite(buzzer, buz);
digitalWrite(led, buz);
for(int i=0;i<samples;i++) // taking samples for calibration
{
```

```
xsample+=analogRead(x);
ysample+=analogRead(y);
zsample+=analogRead(z);
}xsample/=samples; // taking avg for x
ysample/=samples; // taking avg for y
zsample/=samples; // taking avg for z

delay(3000);
lcd.clear();
lcd.print("Calibrated");
delay(1000);
lcd.clear();
lcd.print("Device Ready");
delay(1000);
lcd.clear();
lcd.print(" X Y Z ");
}

void loop()
{
int value1=analogRead(x); // reading x out
int value2=analogRead(y); //reading y out
int value3=analogRead(z); //reading z outint xValue=xsample-value1; // finding change in x
int yValue=ysample-value2; // finding change in y
int zValue=zsample-value3; // finding change in z
```

```
/*displying change in x,y and z axis values over lcd*/
lcd.setCursor(0,1);
lcd.print(xValue);
lcd.setCursor(6,1);
lcd.print(yValue);
lcd.setCursor(12,1);
lcd.print(zValue);
delay(100);

/* comparing change with predefined limits*/
if(xValue < minVal || xValue > maxVal || yValue < minVal || yValue > maxVal ||
zValue < minVal || zValue > maxVal)
{
if(buz == 0)
start=millis(); // timer start
buz=1; // buzzer / led flag activated
}else if(buz == 1) // buzzer flag activated then alerting earthquake
{
lcd.setCursor(0,0);
lcd.print("Earthquake Alert ");
if(millis()>= start+buzTime)
buz=0;
}

else
```

```
{
lcd.clear();
lcd.print(" X Y Z ");
}
digitalWrite(buzzer, buz); // buzzer on and off command
digitalWrite(led, buz); // led on and off command

/*sending values to processing for plot over the graph*/
Serial.print("x=");
Serial.println(xValue);
Serial.print("y=");
Serial.println(yValue);
Serial.print("z=");
Serial.println(zValue);
Serial.println(" $");
}
```